USEI15 – Risk-Aware Shortest Paths

Variáveis

V Número de vértices (estações)

E Número de arestas (ligações dirigidas)

Método Principal

A lógica está concentrada no método `RiskAwareShortestPathServiceImpl.computeRiskAwareShortestPath(Station source, Station target)`.

O cálculo é em `bellmanFordWithNegativeCycles(Graph<Station,Weight>, Station, Station)`.

Bellman-Ford (caminho mínimo com pesos negativos)

Pedaço relevante simplificado:

```
for i in 1..V-1:
   updated = false
   for each edge (u, v, w):
       if dist[u] == INF: continue
       if dist[u] + w < dist[v]:
           dist[v] = dist[u] + w
           parent[v] = u
           updated = true
   if !updated: break
```

Passo	Complexidade	Justificação
Inicialização de `dist[]` e `parent[]`	**O(V)**	Arrays de tamanho V
Ciclo exterior `for i in 1..V-1`	**O(V)**	No máximo V−1 iterações

Passo	Complexidade	Justificação
Ciclo interior sobre todas as arestas	**O(E)**	Cada iteração percorre todas as arestas do grafo
Operações dentro do ciclo interior	**O(1)**	Acessos/atualizações em arrays e comparação de `double`

Complexidade
O(V) + O(V) * O(E) = O(V * E)

Esta é a complexidade clássica do Bellman-Ford em grafos gerais com pesos (incluindo negativos) mas sem ciclos negativos relevantes para o destino (ver pdf das aulas)

Deteção de Ciclos Negativos

Pedaço simplificado:

```
hasNegativeCycle = false
for each edge (u, v, w):
    if dist[u] == INF: continue
    if dist[u] + w < dist[v]:
        hasNegativeCycle = true
        // reconstrução aproximada dos vértices do ciclo usando parent[]
```

Passo	Complexidade	Justificação
Ciclo sobre todas as arestas	**O(E)**	Cada aresta é verificada uma vez
Verificação `dist[u] + w < dist[v]`	**O(1)**	Operação aritmética e comparação
Caminho de subida em `parent[]` (no máximo V)	**O(V)**	Cadeia de predecessores para atingir um vértice do ciclo
Construção do conjunto de vértices do ciclo	**O(V)**	Cada vértice do ciclo é inserido no máximo uma vez

Complexidade :
O(E * V).

Verificar se o ciclo negativo afeta o caminho para o destino

O método `canReachTargetFromAny(...)` executa uma BFS multi-origem a partir dos vértices identificados como pertencentes a ciclos negativos:

```
queue <- todos os vértices de ciclo
while queue not empty:
    cur = queue.poll()
    if cur == target: return true
    for cada adjacente de cur:
        if !visited[adj]: enqueue(adj)
```

Passo	Complexidade	Justificação
Inicialização de `visited[]`	**O(V)**	Array boolean de tamanho V
Inserção inicial dos vértices	**O(K)**	K vértices em ciclos (K ≤ V)
BFS sobre o subgrafo atingível	**O(V+E)**	Cada vértice/aresta visitado no máximo 1x

Complexidade :
O(V + E)

Reconstrução do Caminho e Segmentos de Custo

Uma vez garantido que nenhum ciclo negativo afeta o destino, a reconstrução do caminho e da lista de segmentos é feita percorrendo o vetor `parent[]` e as arestas correspondentes:

Passo	Complexidade	Justificação
Subida em `parent[]` até à origem	**O(V)**	Caminho simples com ≤ V vértices
Conversão de índices em estações	**O(V)**	Uma chamada a `vertex(key)` /vértice
Construção da lista de segmentos	**O(V)**	Um segmento por aresta no caminho

Complexidade Fase 4:
O(V)

Complexidade Total

Agrupando as fases principais:

- Fase 1 (Bellman-Ford): O(V * E)
- Fase 2 (deteção de ciclos negativos): O(E * V)
- Fase 3 (BFS para verificar impacto no destino): O(V + E)
- Fase 4 (reconstrução do caminho): O(V)

Dominado pelos termos O(V * E) e O(V * E):

T(V,E) = O(V * E)

Comparação com Dijkstra

O Dijkstra só funciona bem quando nenhuma aresta tem peso negativo. Se metermos custos negativos nas arestas, ele pode escolher um caminho que afinal não é o melhor.

O Bellman–Ford aguenta pesos negativos sem problema, porque vai "relaxando" todas as arestas várias vezes até ter a melhor
solução. É mais lento, mas neste cenário é o que garante a resposta certa.